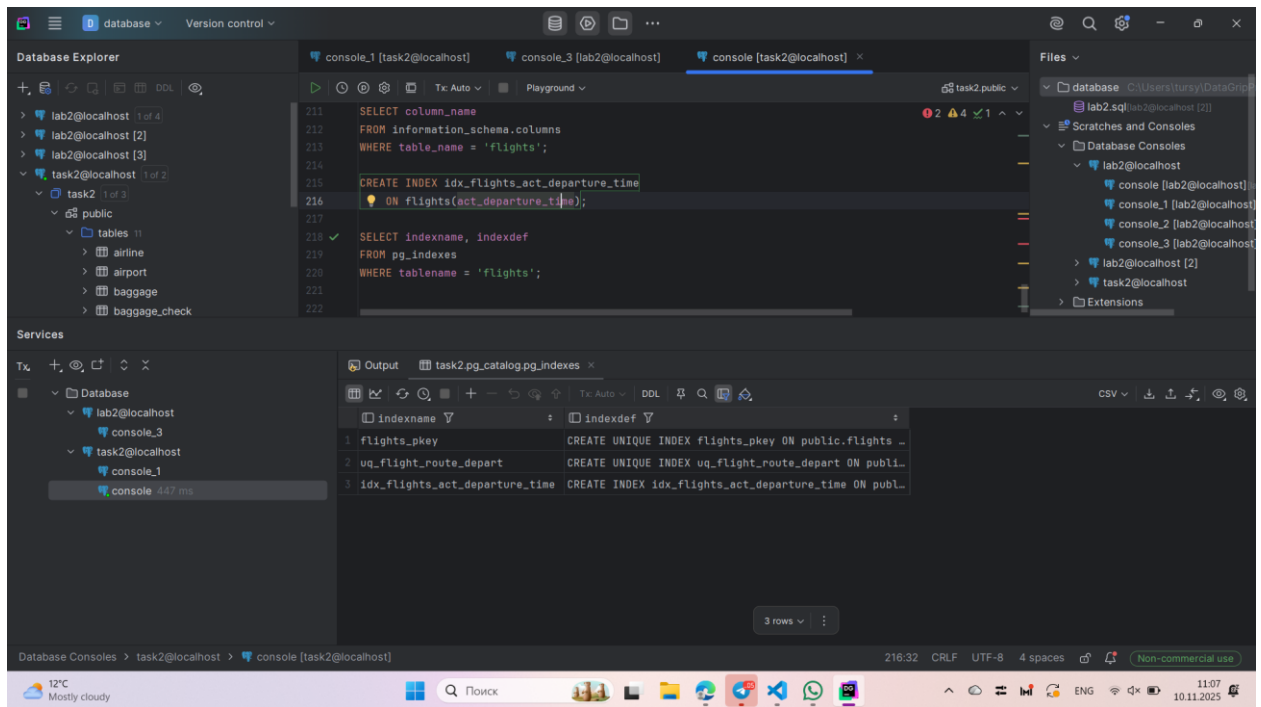
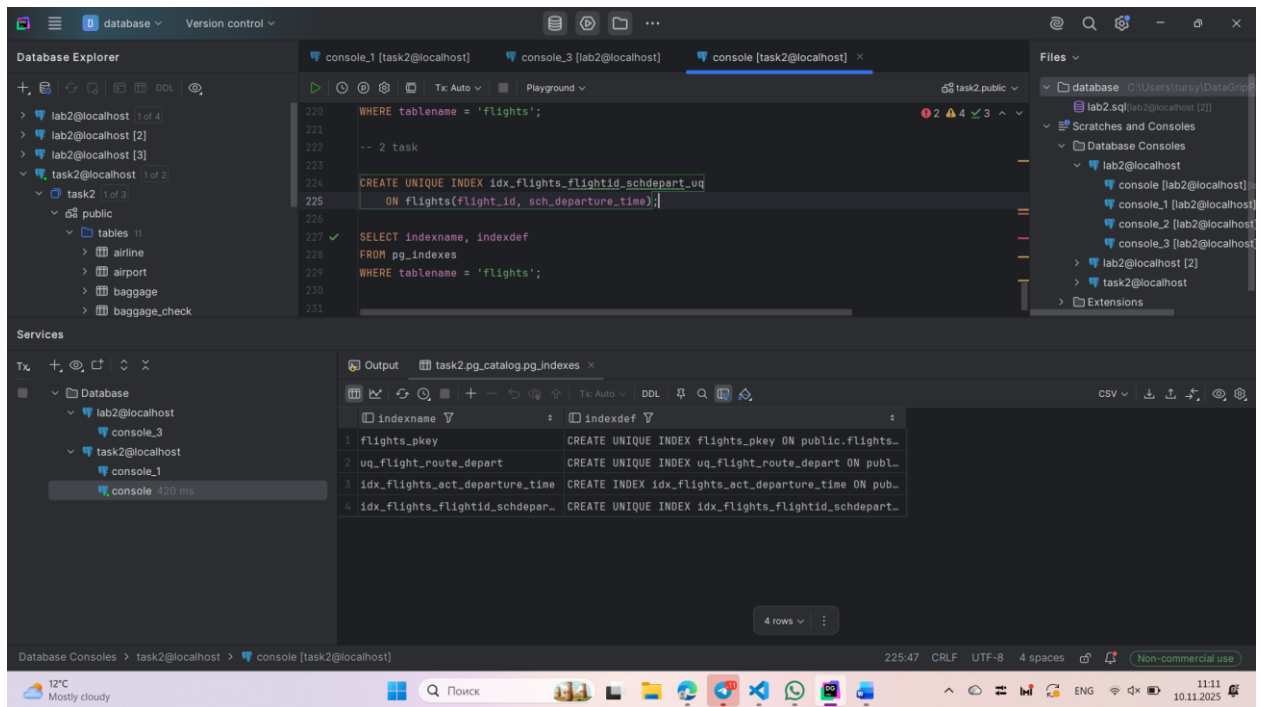


#1



```
CREATE INDEX idx_flights_act_departure_time
ON flights(act_departure_time);
```

#2



```
CREATE UNIQUE INDEX idx_flights_flightid_schdepart_uq
ON flights(flight_id, sch_departure_time);
```

#3

The screenshot shows a database console interface with a dark theme. The top panel displays a SQL query being executed in a console window. The query is as follows:

```
229 WHERE tablename = 'flights';
230
231 --3 task
232
233 CREATE INDEX idx_flights_departing_arriving_airport
234 ON flights(departing_airport_id, arriving_airport_id);
235
236 SELECT indexname, indexdef
237 FROM pg_indexes
238 WHERE tablename = 'flights';
239
240
```

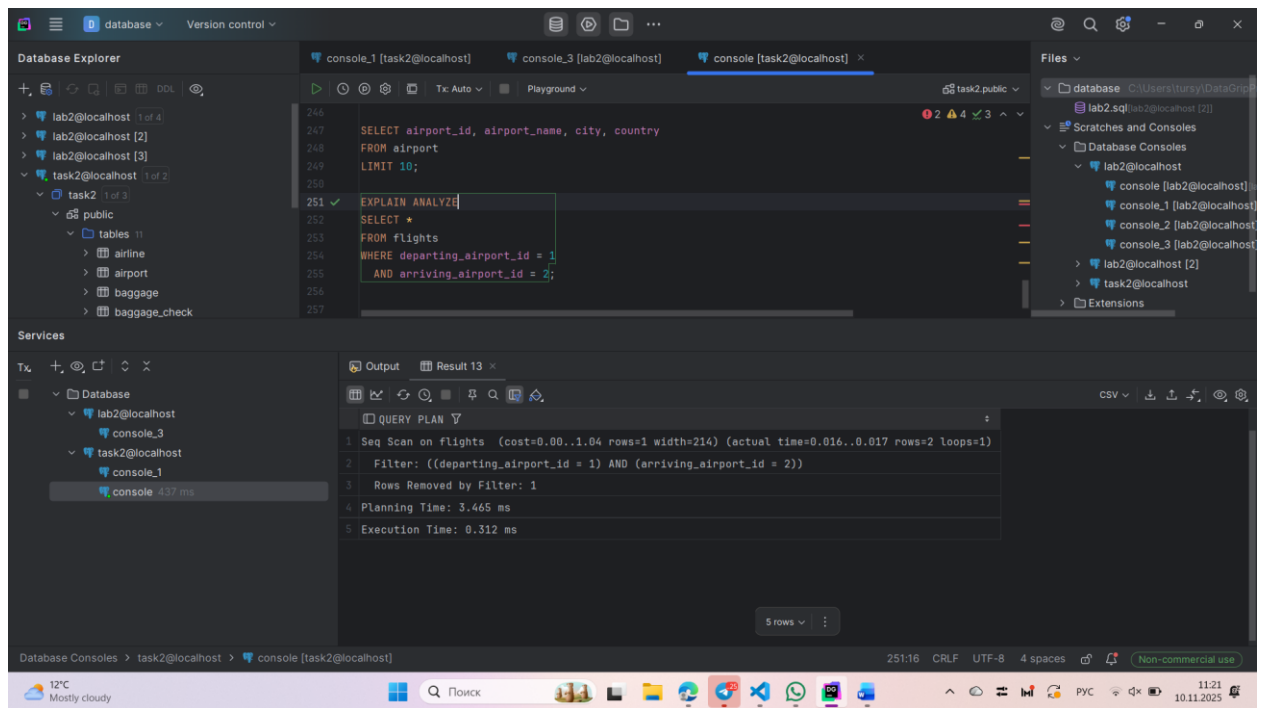
The bottom panel shows the results of the query in a table format. The table has two columns: 'indexname' and 'indexdef'. The results are as follows:

indexname	indexdef
1 flights_pkey	CREATE UNIQUE INDEX flights_pkey ON public.flights_
2 uq_flight_route_depart	CREATE UNIQUE INDEX uq_flight_route_depart ON publ_
3 idx_flights_act_departure_time	CREATE INDEX idx_flights_act_departure_time ON pub_
4 idx_flights_flightid_schdepart_uq	CREATE UNIQUE INDEX idx_flights_flightid_schdepart_
5 idx_flights_departing_arriving_a	CREATE INDEX idx_flights_departing_arriving_airpor_

The interface also includes a 'Database Explorer' on the left side, showing a tree view of the database structure. The bottom status bar indicates the current time is 11:13 on 10.11.2025.

CREATE INDEX idx_flights_departing_arriving_airport
ON flights(departing_airport_id, arriving_airport_id);

#4



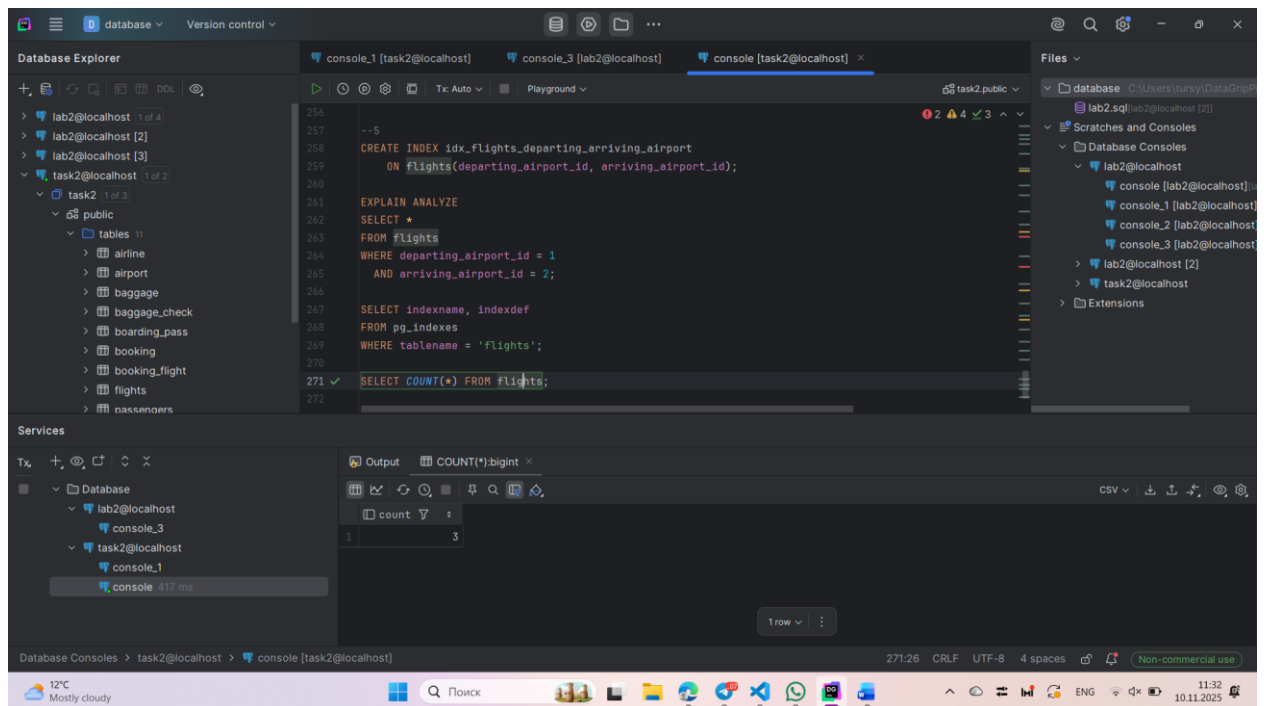
EXPLAIN ANALYZE

SELECT *

FROM flights

WHERE departing_airport_id = 1

AND arriving_airport_id = 2;

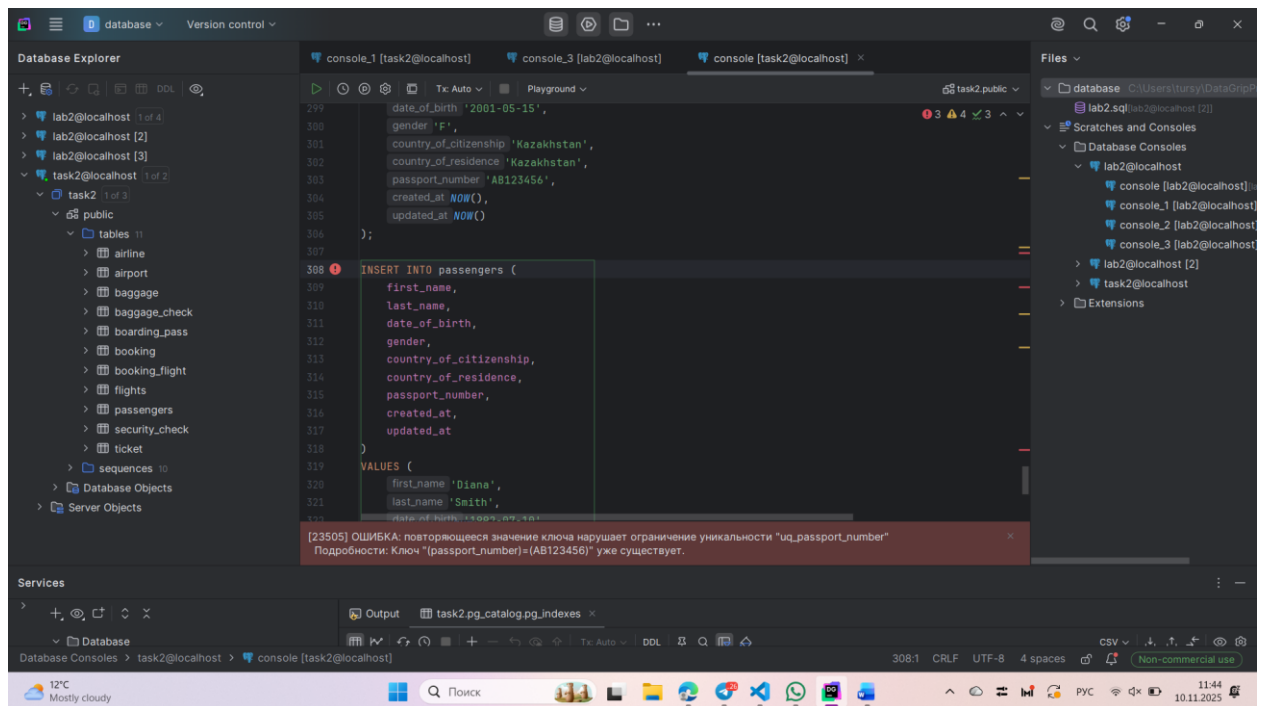


```
CREATE INDEX idx_flights_departing_arriving_airport  
ON flights(departing_airport_id, arriving_airport_id);
```

```
EXPLAIN ANALYZE  
SELECT *  
FROM flights  
WHERE departing_airport_id = 1  
AND arriving_airport_id = 2;
```

```
SELECT indexname, indexdef  
FROM pg_indexes  
WHERE tablename = 'flights';
```

```
SELECT COUNT(*) FROM flights;
```

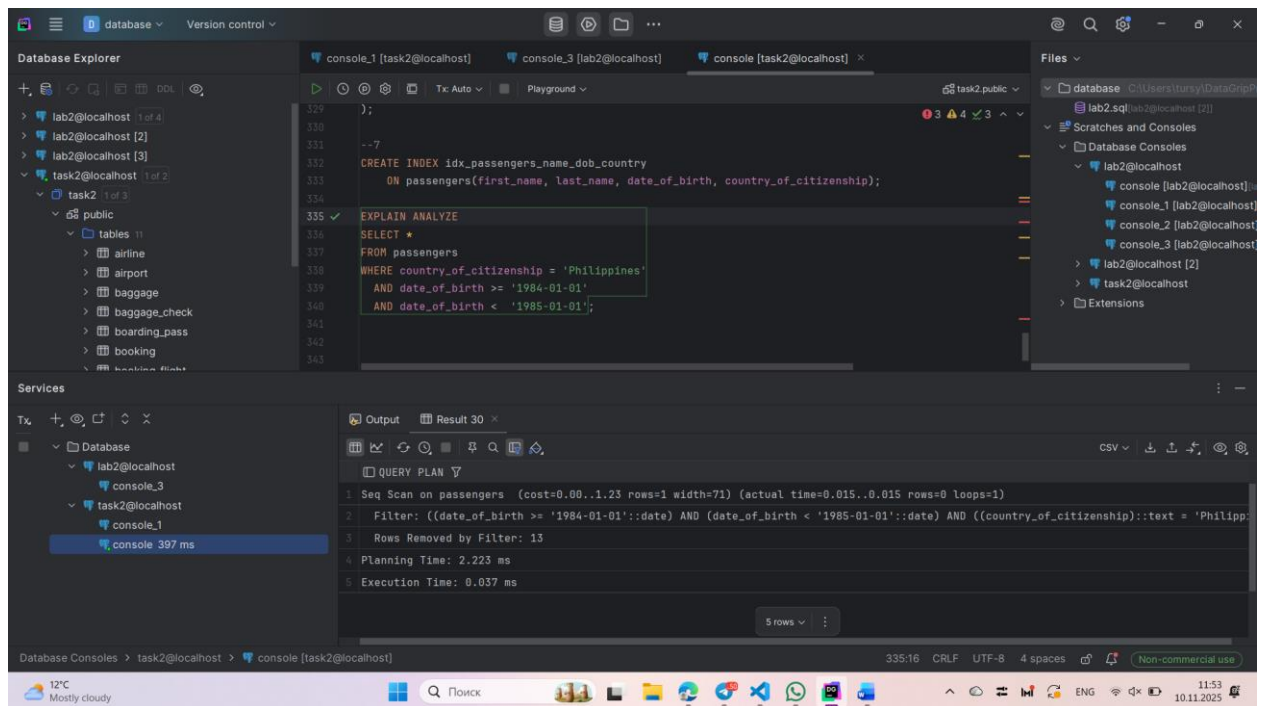


```
INSERT INTO passengers (
  first_name,
  last_name,
  date_of_birth,
  gender,
  country_of_citizenship,
  country_of_residence,
  passport_number,
  created_at,
  updated_at
)
VALUES (
  'Anna',
  'Brown',
  '2001-05-15',
  'F',
  'Kazakhstan',
  'Kazakhstan',
  'AB123456',
  NOW(),
  NOW()
);
```

```
INSERT INTO passengers (
  first_name,
  last_name,
  date_of_birth,
  gender,
  country_of_citizenship,
  country_of_residence,
  passport_number,
  created_at,
  updated_at
)
VALUES (
  'Diana',
  'Smith',
  '1992-07-10',
  'F',
  'Kazakhstan',
  'Kazakhstan',
  'AB123456',
  NOW(),
  NOW()
);
```

The first passenger was added without any problem because the passport number was new. When trying to add another passenger with the same passport number, the system showed an error. This means the unique index works — it doesn't allow duplicate passport numbers.

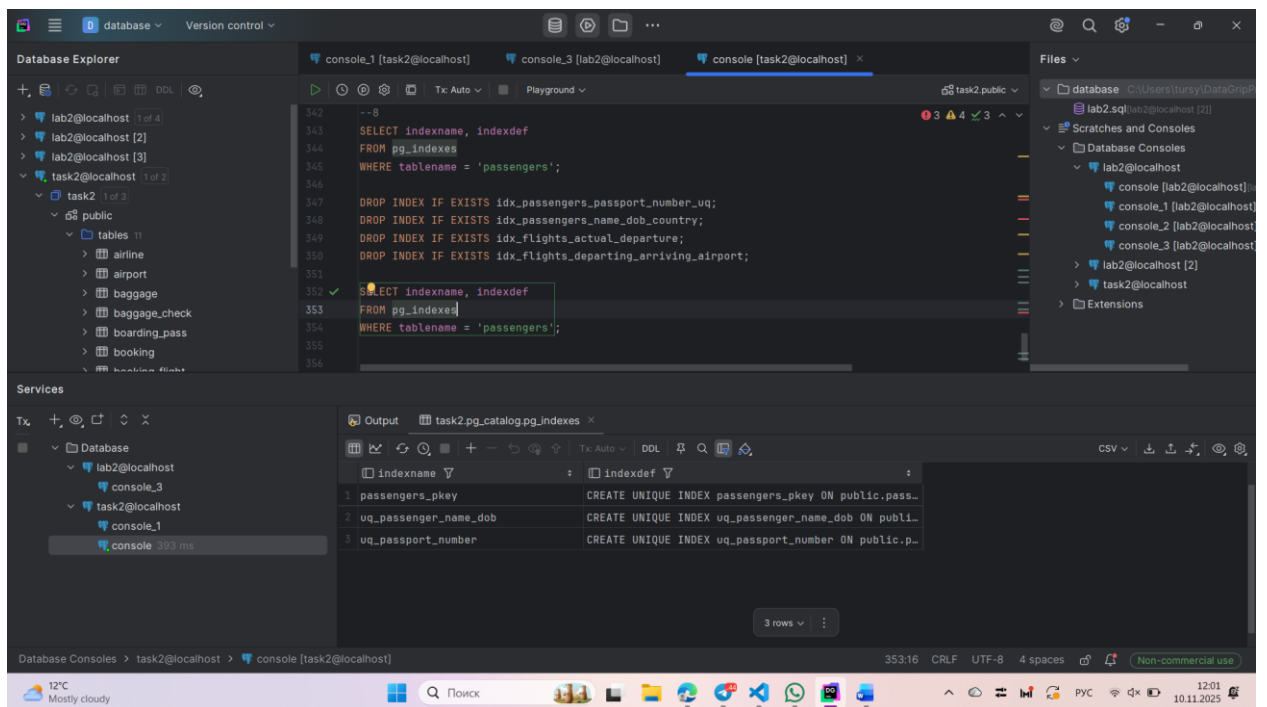
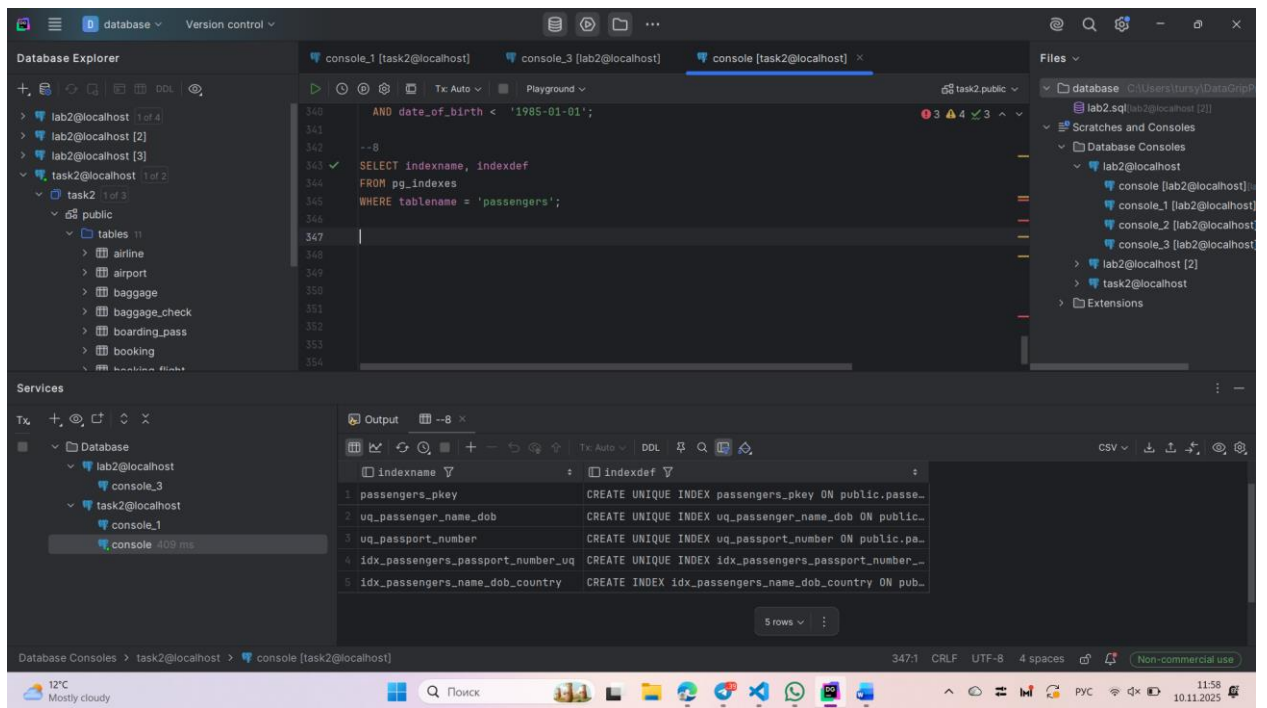
#7



```
EXPLAIN ANALYZE
SELECT *
FROM passengers
WHERE country_of_citizenship = 'Philippines'
      AND date_of_birth >= '1984-01-01'
      AND date_of_birth < '1985-01-01';
```

The query plan shows a **Sequential Scan** on the passengers table, meaning PostgreSQL did not use the created index.

This usually happens when the table is small, or when the filtering conditions (country_of_citizenship and date_of_birth) are not selective enough to benefit from the index. PostgreSQL decides that a sequential scan is faster than using the index in this case. Therefore, the index exists but is not applied during query execution.



After executing the DROP INDEX commands, only the system indexes (passengers_pkey, uq_passenger_name_dob, and uq_passport_number) remained in the table. This confirms that all user-created indexes were successfully delete